MBA em Ciência de Dados e Inteligência Artificial · Turma 2025.1

Previsão interpretável de vida útil remanescente em motores turbofan



Comparação de modelos clássicos de aprendizado de máquina no NASA C-MAPSS FD001

Danrley Nunes Montalvão

Goiânia · 2026

ROTEIRO

Sumário

01

Introdução

Contexto, problema e objetivos

02

Referencial teórico

RUL, C-MAPSS, modelos e SHAP

03

Materiais e métodos

Dados, atributos, validação e métricas

04

Resultados e discussão

Desempenho, erros e interpretabilidade

05

Conclusão

Síntese, limitações e trabalhos futuros

06

Referências

Principais fontes do estudo

Por que estimar a vida útil remanescente?

RUL (Remaining Useful Life): intervalo provável em que um equipamento ainda opera antes de exigir reparo ou substituição.

- Programar inspeções com base quantitativa
- Distribuir recursos de manutenção
- Reduzir paradas não planejadas

Previsão cedo demais

Antecipa trocas e desperdiça vida útil: custo.

Previsão tarde demais

Mantém componente degradado em serviço: risco à segurança.

Problema de pesquisa

> Entre modelos clássicos de aprendizado de máquina treinados com atributos de sensores, **qual oferece o melhor equilíbrio** entre precisão, risco de erros tardios, explicabilidade e viabilidade de implantação no FD001?



Precisão

RMSE e MAE da RUL



Segurança

Penalizar erros tardios



Explicabilidade

Sinais que sustentam a previsão



Implantação

Custo e tamanho do modelo

Objetivos e hipótese

OBJETIVO GERAL

Comparar Ridge, Random Forest, XGBoost, LightGBM e Stacking na previsão da RUL.

OBJETIVOS ESPECÍFICOS

1. Estruturar processamento sem vazamento entre motores
2. Avaliar RMSE, MAE e PHM Score
3. Analisar a assimetria dos erros
4. Explicar o melhor modelo com SHAP
5. Discutir limitações e impactos na manutenção

Hipótese

Árvores com boosting capturam melhor a não linearidade da degradação que o baseline linear. O Stacking não traz ganho suficiente para compensar sua complexidade.

O benchmark NASA C-MAPSS

Subset	Treino	Teste	Condições	Modos de falha
FD001	100	100	1	1
FD002	260	259	6	1
FD003	100	100	1	2
FD004	249	248	6	2

Motores por subconjunto. FD001 (destacado) foi modelado; os demais servem de contexto.

Cada observação

ID do motor · ciclo · 3 configurações operacionais · 21 sensores

Treino × teste

Treino vai até a falha; teste é cortado antes, com RUL real em arquivo separado

Por que FD001?

1 condição e 1 modo de falha: menos fatores de confusão

Cinco modelos em complexidade crescente

linear L2

Ridge

Baseline regularizado; simples, mas limitado para degradação não linear.

bagging

Random Forest

Agrega árvores com reamostragem; reduz variância.

boosting

XGBoost

Árvores corrigem resíduos em sequência, com regularização.

boosting

LightGBM

Divisões mais rápidas; acelera bases de alta dimensão.

meta-ensemble

Stacking

RF + XGB + LGB combinados por meta-modelo Ridge.

Todos recebem os **mesmos 84 atributos**. LSTM ficou fora do escopo: o foco é menor complexidade e explicações por sensor.

Interpretabilidade com SHAP

O SHAP nos permite entender como as variáveis de entrada contribuem para a previsão do modelo.

Local

Por que este motor recebeu esta RUL?

Global

Quais sensores mais influenciam o modelo?

Relevante em Manutenção

A recomendação deve ser acompanhada dos sinais que a sustentam

⚠ SHAP não prova causalidade, verifica se o modelo usa variáveis tecnicamente coerentes.

Visão geral do pipeline experimental

01
Dados
FD001: 100 motores de treino e 100 de teste

→

02
Alvo
RUL com teto de 125 ciclos só no treino

→

03
Atributos
14 sensores × 6 estatísticas = 84

→

04
Validação
GroupKFold (5) + Optuna, 100 tentativas

→

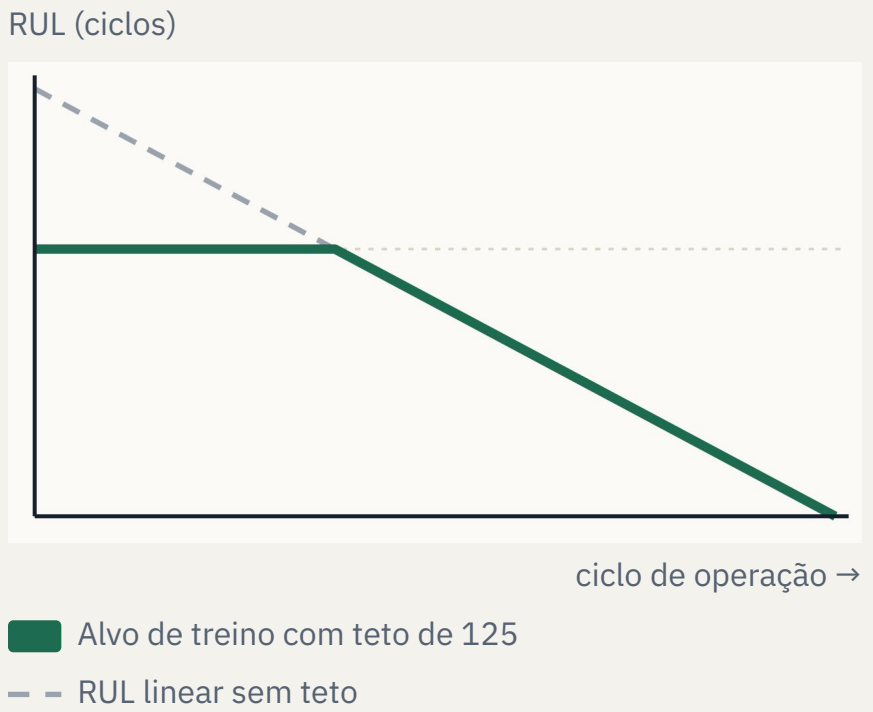
05
Teste + SHAP
Teste único com 100 motores e explicações

- Unidade de independência: o motor, não a linha
- Teste oficial isolado e consultado uma única vez
- Semente 42 e execução determinística

Preparação do alvo: RUL com teto

$$RUL_{cap}(i,t) = \min[T(i) - t,\ 125]$$

- RUL de treino = último ciclo – ciclo atual
- Teto só no treino: fase saudável é indistinguível
- Teste mantém a RUL real: 7 a 145 ciclos (média 75,5)
- 7 sensores sem variância removidos (1, 5, 6, 10, 16, 18, 19) → 14 informativos



Engenharia de atributos

14 sensores × 6 estatísticas = 84 atributos em janela móvel de 30 ciclos

Janelas de 10, 20, 30 e 50 comparadas por correlação de Spearman: 30 foi o melhor compromisso.

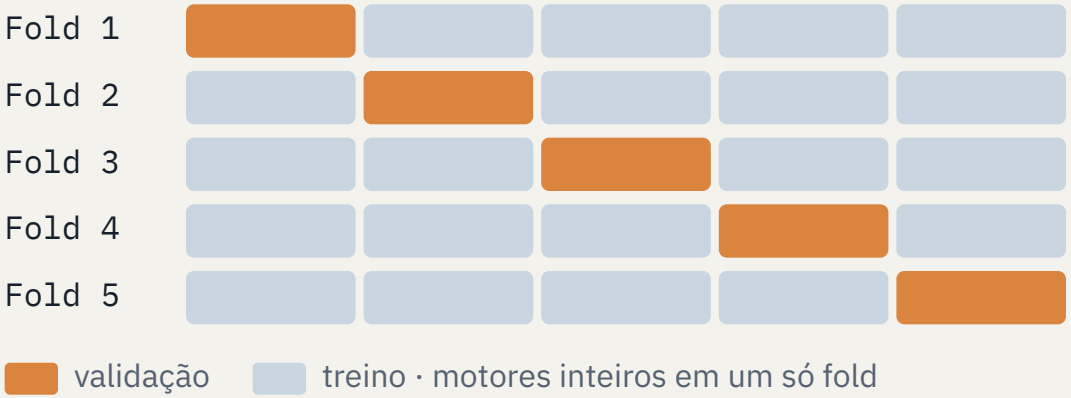
Estatística	Nome lógico	Informação
Média	rolling_mean_30	nível local do sinal
Desvio padrão	rolling_std_30	instabilidade na janela
Mínimo	rolling_min_30	menor valor recente
Máximo	rolling_max_30	maior valor recente
Inclinação	rolling_slope_30	taxa local de mudança
Delta basal	delta_baseline	desvio desde o início

TRECHO DE IMPLEMENTAÇÃO · PANDAS

```
windows_test = [10, 20, 30, 50]
sensores_ref_w = ["sensor_4", "sensor_11", "sensor_14", "sensor_2"]

max_c_w = train_norm.groupby("unit_id")["cycle"].transform("max")
rul_train_w = (max_c_w - train_norm["cycle"]).clip(upper=RUL_CAP)
for sensor in sensores_ref_w:
    corrs = []
    for w in windows_test:
        media = train_norm.groupby("unit_id")[sensor].transform(
            lambda x, ww=w: x.rolling(ww, min_periods=1).mean()
        )
        rho, _ = _stats2.spearmanr(media, rul_train_w)
        corrs.append(rho)
    linha = f"{sensor:<15s} | " + " | ".join(f"{r:+.3f}" for r in corrs)
    # Marca a janela 30 como referência
    print(linha + "  <- janela escolhida" if 30 in windows_test else
    linha)
train_feat = rolling_features(train_norm, SENSOR_COLS, window=ROLLING_WINDOW)
test_feat  = rolling_features(test_norm, SENSOR_COLS, window=ROLLING_WINDOW)
feat_cols = [c for c in train_feat.columns if c not in train_raw.columns]
```

Validação sem vazamento entre motores



- GroupKFold com 5 folds agrupados por unit_id
- Optuna: 100 tentativas por modelo
- Objetivo: RMSE médio de validação

TRECHO DE IMPLEMENTAÇÃO · SCIKIT-LEARN + OPTUNA

```
cv = GroupKFold(n_splits=5)
study_ridge = optuna.create_study(
    direction="minimize",
    sampler=optuna.samplers.TPESampler(seed=SEED),
    study_name="ridge_FD001",
)
study_ridge.optimize(ridge_objective, n_trials=N_TRIALS,
show_progress_bar=True)

best_ridge_params = study_ridge.best_params
```

Métricas: erro médio e risco assimétrico

RMSE · métrica principal

$$\sqrt{[(1/n)\ \Sigma(\hat{y}_i - y_i)^2]}$$

Penaliza mais os erros grandes

MAE

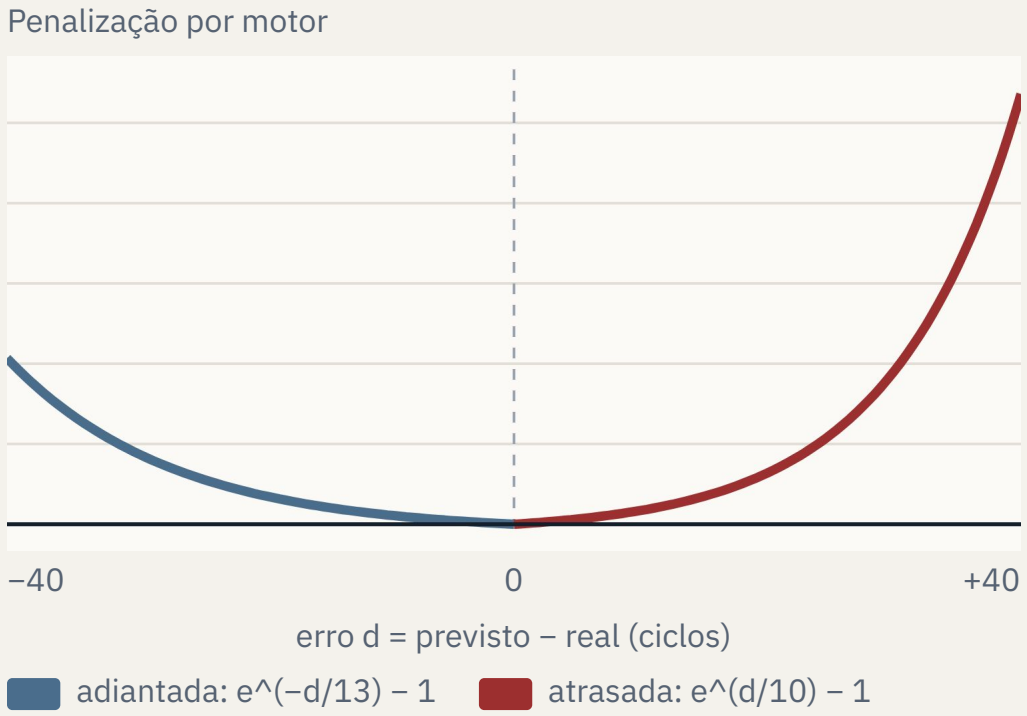
$$(1/n)\ \Sigma|\hat{y}_i - y_i|$$

Erro médio direto, em ciclos

PHM Score · assimétrico

$$\Sigma\ \text{penalizações (menor = melhor)}$$

Atraso cresce mais rápido: risco de superestimar a vida



Desempenho dos modelos no FD001

Modelo	CV RMSE	CV MAE	Teste RMSE	Teste MAE	PHM Score
Ridge	16,81 ± 0,51	13,73 ± 0,69	18,59	14,85	501,2
Random Forest	12,67 ± 1,02	9,11 ± 0,81	13,42	10,02	252,6
XGBoost	12,27 ± 0,67	8,70 ± 0,58	12,50	9,04	206,3
LightGBM	12,40 ± 0,78	8,73 ± 0,65	12,79	9,27	222,4
Stacking	12,23 ± 0,70	8,55 ± 0,62	12,74	9,18	220,5

Menor RMSE no teste
12,50

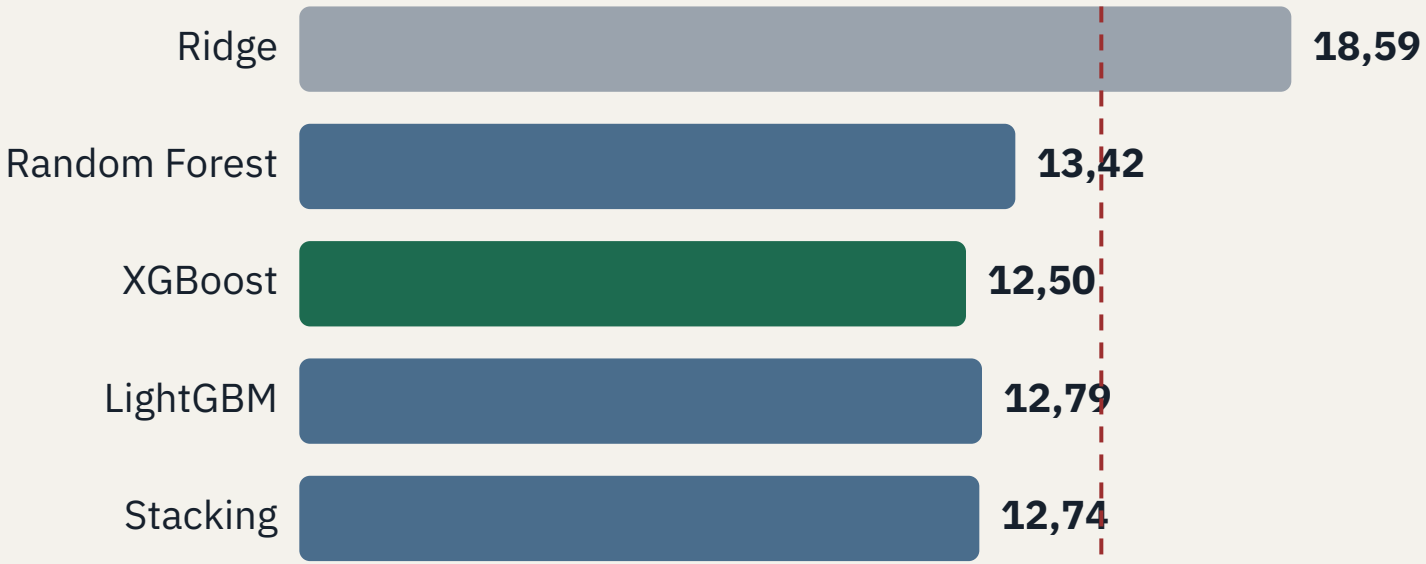
Menor MAE no teste
9,04

Menor PHM Score
206,3

XGBoost venceu nas três métricas de teste

RMSE no teste: boosting à frente

RMSE no conjunto de teste (ciclos)



linha tracejada: critério interno do estudo = 15 ciclos

−32,8%
RMSE vs. Ridge (−6,09 ciclos)

−58,8%
PHM Score vs. Ridge

CV ≈ teste no boosting: sem sinal de sobreajuste relevante

STACKING × XGBOOST

Mais complexidade, sem ganho

12,74 vs 12,50

RMSE no teste: Stacking × XGBoost

81 MB vs 840 KB

Tamanho do artefato serializado

O Stacking liderou a validação (12,23), mas no teste adicionou custo de armazenamento, carregamento e manutenção **sem ganho preditivo verificável**.

Direção dos erros e penalização

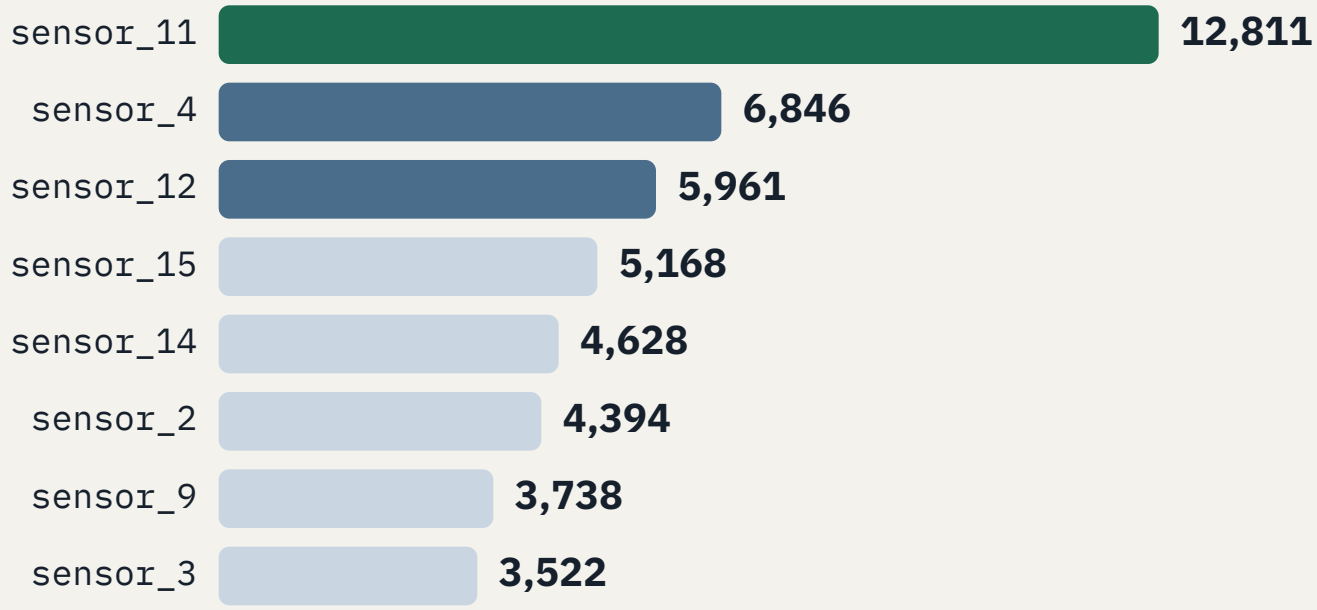
Modelo	% adiantado	MAE adiantado	% atrasado	MAE atrasado	Viés	PHM
Ridge	47	16,2	53	13,7	−0,3	501,2
Random Forest	37	12,6	63	8,5	+0,7	252,6
XGBoost	37	12,6	63	7,0	−0,3	206,3
LightGBM	38	12,0	62	7,6	+0,1	222,4
Stacking	33	13,7	67	7,0	+0,1	220,5

63% atrasadas, mas MAE atrasado de apenas 7,0 ciclos

Viés médio de −0,3 ciclo: praticamente centrado

Stacking: 67% atrasadas → PHM de 220,5

Quais sensores guiam o XGBoost?



Média agregada de |SHAP| (6 atributos por sensor), conjunto de teste

Sensor 11 · Ps30
Pressão estática na saída do compressor de alta pressão

Sensor 4 · T50
Temperatura total na saída da turbina de baixa pressão

Sensor 12 · Phi
Razão entre fluxo de combustível e Ps30

Explicação local: dois motores

Motor 81

Previsto	Real
8	8

Sensores 11 e 4 empurraram a previsão para baixo: degradação avançada.

Motor 83

Previsto	Real
125,7	137

Erro de −11,3 (conservador); o perfil do sensor 11 elevou a RUL.

A explicação local transforma um número em **narrativa técnica verificável**: quais sinais, em que direção e com que consequência.

Conclusões

MODELO RECOMENDADO

XGBoost: melhor equilíbrio entre precisão, risco, custo e explicabilidade

12,50	9,04	206,3
RMSE	MAE	PHM

- Hipótese confirmada: árvores superam o baseline Ridge
- Stacking não compensa a complexidade (81 MB × 840 KB)
- SHAP: sensores 11, 4 e 12 — pressão, temperatura e razão de fluxo
- Maiores erros foram conservadores; nenhum tardio > 30 ciclos
- Adoção condicionada a múltiplos regimes, dados reais e incerteza

Trabalhos futuros

- FD002–FD004 com normalização por regime
- Comparar com LSTM e Transformers
- Intervalos por predição conformal
- Monitoramento de deriva dos dados
- Políticas de manutenção por custo e risco

Próximo passo natural: sair do benchmark controlado rumo a múltiplos regimes, incerteza calibrada e integração com a decisão de manutenção.

Principais referências

SAXENA, A. et al. Damage Propagation Modeling for Aircraft Engine Run-to-Failure Simulation. In: Int. Conf. on PHM. IEEE, 2008.

RAMASSO, E.; SAXENA, A. Performance Benchmarking and Analysis of Prognostic Methods for CMAPSS Datasets. IJPHM, v. 5, n. 2, 2014.

SOUALHI, M.; NGUYEN, K. T. P.; MEDJAHER, K. Explainable RUL Estimation of Turbofan Engines... EAAI, v. 133, 2024.

NGUYEN, D. V. et al. Modeling and Prediction of RUL for Maintenance Scheduling Optimization of a Car Fleet. IJPE, v. 15, n. 9, 2019.

CHEN, T.; GUESTRIN, C. XGBoost: A Scalable Tree Boosting System. In: ACM SIGKDD, 2016.

KE, G. et al. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In: NeurIPS 30, 2017.

LUNDBERG, S. M.; LEE, S.-I. A Unified Approach to Interpreting Model Predictions. In: NeurIPS 30, 2017.

AKIBA, T. et al. Optuna: A Next-generation Hyperparameter Optimization Framework. In: ACM SIGKDD, 2019.

Lista completa (15 referências) no artigo.

Obrigado!

Perguntas e contribuições

danrley.1260@gmail.com

